

NetSentinel — Condensed Project Context

Purpose: Feed this to GPT Astra (or any LLM) for full project context in one file. **Generated:** 2026-09-10 | **Project:** SIH 2026 — AI/ML NIDS **Team:** 2–3 devs + AI (Claude, Antigravity, GPT Astra) **Stage:** Demo-ready prototype → Grand Finale (submit 20 Sep)

1. What NetSentinel IS

A **passive AI Network Intrusion Detection System** for unidirectional traffic behind data diodes (nuclear, defence, SCADA). No DPI, no signatures, CPU-only, detection-only.

Two tiers:

Tier	Status	What it does
Tier 1	✅ BUILT, 6/6 models load	6 ONNX expert models for known threat classes
Tier 2	🔴 PARTLY BUILT	Inspector–Sentry cascade for LOTS/LSA detection (the novel contribution)

2. Tier 1 — The 6 Expert Models (ALL BUILT)

#	Model	Architecture	Training Data	Performance	Status
A	DDoS	XGBoost → ONNX	CIC-DDoS2019 (2.5M flows, 59 CIC features)	99.3% F1	✅ Working
B	DGA	CNN-BiLSTM	UMUDGA + Tranco (1.2M domains)	93.6% 3-class	✅ Working
C	C2 Beacon	BiLSTM+FFT dual-branch	CTU-13 (13 botnet scenarios)	93.5% accuracy	✅ Working (genuine novel contribution)
D	Encrypted Traffic	FT-Transformer	ISCX-VPN-NonVPN (150K flows)	88% 14-class	✅ Working
E	Port Scan	XGBoost → ONNX	UNSW-NB15 (39 features)	96.4% F1 (reported)	⚠️ 0 detections on CIC data (wrong dataset)
F	Exfiltration	VAE anomaly	CIC-Bell-DNS-EXF-2021 (24 DNS features)	91.2% AUC	⚠️ ~50% FP (scikit-learn version mismatch)

Feature Extractors (3 non-overlapping schemas)

- **59 CIC-IDS features** → DDoS, Encrypted, (Port Scan wrongly fed this)
- **39 UNSW-NB15 features** → Port Scan (correct schema)
- **24 DNS-lexical features** → Exfiltration VAE

Pipeline Flow

```

PCAP / Live Capture / Simulator
→ PacketProcessor (pcap_reader.py)
  ↳ FlowExtractor (59 CIC features) → "flow" events
  ↳ DNSExtractor → "dns" events
  ↳ SessionBuilder (100-flow windows) → "session" events
→ FlowAnalyzer (routes to correct models)
  ↳ "flow" → DDoS + ETT + PortScan
  ↳ "dns" → DGA + Exfiltration
  ↳ "session" → C2 Beacon
→ AlertManager (MITRE mapping + severity + dedup)
→ WebSocket → React Dashboard

```

Real-World Test (Friday-WorkingHours.pcap, 8.8GB)

- **32,266 alerts** across all 6 classes
- 42.5 flows/sec, 23ms latency, ~280MB RAM
- All 6 models load in 0.61s

3. Tier 2 — Inspector–Sentry (THE CORE INNOVATION)

The Problem

Modern intrusions use **Living off Trusted Sites (LOTS/LSA)** — every hop rides domains nobody can block (ip-api → GitHub → Telegram → Google Drive). Each request alone looks normal. **The signal is the cross-service category-transition SEQUENCE.**

Architecture (Built Components)

```

ALL HOST TRAFFIC
↓
COMMISSIONING (tiered length) → INSPECTOR (expensive E-GraphSAGE + Transformer)
↓                               ↓
threat found? —yes→ HELD on Inspector (never demoted) ← RETENTION RULE
↓ no
CLEARED → demoted to SENTRY (cheap distilled model, always-on)
↓
Re-escalation triggers:
(a) anomaly — off-path category transition
(b) random sample — unpredictable, can't be timed
(c) behavioural change — drift detection

```

↓

Back to INSPECTOR to confirm

↓

benign/drift → re-baseline → Sentry

malicious → LLM verdict → ALERT (chain summary + MITRE + confidence)

What's Actually Built in Code (v2/ directory)

Component	File	Status
Inspector model (E-GraphSAGE + Transformer AE)	<code>netsentinel_v2/models.py</code> — <code>Inspector</code> class	✔ Built, trained, measured
Sentry model (distilled, GRU, no hop-2)	<code>netsentinel_v2/models.py</code> — <code>Sentry</code> class	✔ Built, trained, measured
Deferral Head (router C)	<code>netsentinel_v2/models.py</code> — <code>DeferralHead</code> class	✔ Built
Per-host baseline (Ledoit-Wolf + KMeans)	<code>netsentinel_v2/baseline.py</code> — <code>HostBaseline</code>	✔ Built
Service Category Resolver	<code>netsentinel_v2/categories.py</code> + <code>zeek_loader.py</code>	✔ Built (9 categories)
Synthetic data generator (with hard negatives)	<code>netsentinel_v2/synth.py</code>	✔ Built
Zeek log loader (real traffic)	<code>netsentinel_v2/zeek_loader.py</code>	✔ Built
LANL cyber1 loader (real red team)	<code>netsentinel_v2/lanl_loader.py</code>	✔ Built
Router comparison harness	<code>run_experiment.py</code>	✔ Built
Distribution shift test	<code>shift_test.py</code>	✔ Built
Real traffic training	<code>train_real.py</code>	✔ Built
Interactive demo app	<code>inspector_centry/App.tsx</code> (48KB React component)	✔ Built
Escalation loop / retention rule runtime	—	✗ NOT BUILT
Random sampling scheduler	—	✗ NOT BUILT
Rolling re-calibration	—	✗ NOT BUILT
LLM verdict integration	—	✗ NOT BUILT

Service Category Taxonomy (9 categories)

Recon_API — ip-api.com, ipify.org (victim fingerprinting)

Code_Repo_Paste — github, pastebin (stager pull)

Messaging_API — telegram, discord (C2 polling)

Cloud_Storage — drive.google.com, dropbox (exfil T1567)

Browse — general web

Sync — OneDrive/Dropbox agents

CI_CD — build runners, artifact stores

Internal — east-west traffic (OT/diode generalisation)

Unknown_External — resolver failed (ECH/DoH/raw IP)

Edge Features (10 per host-category-window)

log_n_flows, log_bytes_up, log_bytes_down, egress_asymmetry,

iat_cv, ks_uniform, ks_exponential, fft_prominence,

distinct_endpoint_ratio, log_duration_mean

Measured Results (Post-Bugfix, 3 Seeds)

Synthetic (300 hosts, 24 days):

Router	AUC vs Inspector	Recall @5% budget
A — distilled detector	0.876 ← BEST	37.1% ± 0.9
B — encoder+Mahalanobis	0.818 ± 0.046	32.8% ± 6.9
C — deferral head	0.850	38.3% ± 7.0

Compression: 205,546 → 14,992 params (13.7×)

LANL cyber1 (real traffic, real red team, 612 hosts × 13 days):

Metric	Value
Cascade AUC (router A vs Inspector)	0.992 ± 0.003
Cascade recall @5% budget	96.9% ± 1.1 (agreement with teacher, NOT detection)
Within-host AUC (the real detection number)	0.557 ± 0.013 — CHANCE
Global AUC (confounded — between-host effect)	0.745

Key finding: The cascade WORKS (Router A tracks Inspector excellently). Detection on LANL is CHANCE because lateral movement uses access the host already has. The global AUC is a confound — attacked hosts are busier (density 0.811 vs 0.530).

Shift test (World A → World B):

- Mechanism transfers: AUC drops only -3.3%
- **Calibration does NOT transfer:** flag rate 6.82% in own test period (target 1.0%)
- Threshold decays over time — rolling re-calibration is mandatory

4. Frontend (React + Vite + TypeScript)

15 components including:

- Real-time alert feed with WebSocket
- 3D threat graph (Three.js / WebGL force-directed)
- MITRE ATT&CK 14-tactic heatmap
- Model status cards with accuracy rings
- Traffic charts (Recharts)
- Glassmorphism design system

5. Known Bugs & Issues

Issue	Severity	Fix
Exfil VAE ~50% FP	Critical	<code>pip install scikit-learn==1.6.1</code> or re-save scaler
Port scan 0 detections	Critical	Needs scan-heavy dataset (UNSW-NB15 or IoT-23), not a code bug
92% features drifted vs CICFlowMeter	Critical	3 bugs fixed; <code>ks_summary.txt</code> is STALE — rerun <code>scripts/covariate_shift.py</code>
2/6 evidence panels wrong visualization	Medium	Exfil panel shows byte-ratio but model uses DNS-lexical
No persistence (in-memory alerts)	Low	—
No authentication on API	Low	—
No SIEM/SOAR connectors	Low	—

6. Networking / Capture — BLOCKED ISSUES

The Cloudflare / DoH Problem

During the 12-day local capture (started ~5 Sep), three defects were found on day 1:

1. **Snaplen 160 destroyed ALL TLS hostnames** — 100% of 4,539 ClientHellos truncated, 0 SNI hostnames recovered. Fixed: snaplen → 512.
2. **61% of encrypted traffic is QUIC** — SNI inside encrypted Initial packet. Fix: disable QUIC in Chrome/Edge via Group Policy.
3. **DNS-over-HTTPS (Cloudflare)** hides the fallback naming path — 55% of DNS queries were FOR DoH resolvers (`chrome.cloudflare-dns.com`). Fix: disable DoH in browser policies.

All three are fixed in `capture_service.ps1` and `start_capture.ps1`. The broken first 21 hours are kept as evidence.

What This Means for the Network Scan

The networking part (local network scan + live capture) was blocked because:

- Cloudflare DoH was eating all the DNS queries → resolver blind
- QUIC was hiding 61% of browsing hostnames → category resolver can't work
- Without hostnames, the Service Category Resolver puts everything in `Unknown_External` → the cross-service sequence hypothesis CANNOT be tested

After applying the fixes: the capture should produce real hostnames, and the Zeek loader (`zeek_loader.py`) + `train_real.py` can process it.

7. WHAT TO DO NEXT — Future Goals & Tasks

● IMMEDIATE (Before 20 Sep Submission)

A. Rolling Re-Calibration (1 dev, 9–12 Sep)

The threshold decays from the day it's set (6.82% flag rate vs 1.0% target). This is the one feature your own evidence demands.

- ☐ Implement rolling window re-estimate of Inspector threshold (hold last N days, re-take 99th percentile)
- ☐ Guard it: cap flag-rate movement per re-calibration cycle
- ☐ Measure: re-run `shift_test.py` and `train_real.py --lanl` with re-calibration ON
- ☐ Write into ARCHITECTURE_V2 §4 as a real component

B. Chain Injection on Real Capture (1–2 devs, 9–16 Sep)

The ONLY route to a detection number you can defend.

- ☐ Script LOTS chains over captured benign base: `Recon_API → Code_Repo_Paste → Messaging_API → Cloud_Storage` at realistic beacon intervals
- ☐ Include hard negatives (DevOps hosts walking the same category sequence with human timing)
- ☐ Inject as Zeek-format rows so `zeek_loader.load_dir()` reads them unchanged
- ☐ Evaluate with `train_real.py --seeds 3`, report **within-host AUC** (not global)

C. Local Network Scan & Dataset Creation

This is the piece that was blocked by the Cloudflare/QUIC/DoH issues.

Steps to build:

1. Fix the capture environment (already done in scripts):

- Disable QUIC: `Set-ItemProperty "HKLM:\SOFTWARE\Policies\Google\Chrome" -Name "QuicAllowed" -Value 0`
- Disable DoH: `Set-ItemProperty "HKLM:\SOFTWARE\Policies\Google\Chrome" -Name "DnsOverHttpsMode" -Value "off"`
- Snaplen 512 in capture scripts

2. Run the local capture (12 days, multiple machines):

```
cd D:</span>1_sih26#2\netsentinel-main\v2
.</span>start_capture.ps1 -Interface <n> -OutDir D:</span>capture -MaxGB 40
```

3. Convert to Zeek logs (in WSL):

```
./zeekify.sh /mnt/d/capture /mnt/d/capture-zeek
```

4. Verify resolver works:

```
.\.venv\Scripts\python.exe train_real.py --data D:\capture-zeek --seeds 3
```

Look at **resolver hit rate** — target is ~77% (WRCCDC baseline).

5. Create the dataset by overlaying injected LOTS chains onto the benign capture

6. Final run on full 12 days with `--seeds 3`

D. Submission Document

- ☐ Lead with cascade result; state the LANL negative yourself
- ☐ Correct positioning: §9's OT reframe is NOT evidence-backed
- ☐ State prior art: Inspector = GraphIDS (arXiv:2509.16625)
- ☐ Delete stale files: `lanl_results.json`, `lanl_src.json`, `lanl_dst.json`, `lanl_both.json`

● MEDIUM-TERM (Post-Submission, for Grand Finale)

E. Build the Escalation Runtime

The routers are scorers, not yet the architecture. Build the actual runtime:

- ☐ Escalation loop: Sentry flags → host returns to Inspector
- ☐ Retention rule: threats during commissioning stay on Inspector forever
- ☐ Random sampling scheduler (anti-poisoning mechanism)
- ☐ Behavioural-drift vs taxonomy-drift separation

F. LLM Verdict Integration

- ☐ Wire up an LLM (GPT-4 / Claude) to produce chain narratives from confirmed alerts
- ☐ Output: MITRE technique + confidence + human-readable summary
- ☐ Display in the Inspector-Sentry demo app

G. Fix Tier 1 Known Issues

- ☐ Port scan model: retrain on IoT-23 (213M labelled port scan flows) or real scan-heavy data
- ☐ Exfil VAE: pin scikit-learn==1.6.1 or re-export scaler
- ☐ Evidence panels: fix exfil panel (show DNS-lexical reconstruction error, not byte-ratio)
- ☐ Rerun covariate shift analysis post-fix

●

LONG-TERM (Future Work)

- ☐ SHAP explainability for all 6 models
- ☐ Meta-classifier MLP ensemble fusion across experts
- ☐ Alert persistence (database backend)
- ☐ Authentication on API
- ☐ SIEM/SOAR connectors (Splunk, ELK, QRadar)
- ☐ Public LOTS chain dataset — the 124-dataset survey found none; building one is a contribution
- ☐ DARPA TC/OpTC validation — real adversary sequences for sequence-hypothesis testing
- ☐ Caldera/Mythic emulation — the malicious half of the dataset

8. What NOT to Claim

- ✖ "State-of-the-art accuracy" (88% vs 99.93% with better features)
- ✖ "Novel transformer approach" (transformers for traffic are well-known)
- ✖ "We replace EDR" (strictly worse on host signal)
- ✖ "≥95% recall at 5% budget" (measured: 37.1%)
- ✖ "We detect attacks on LANL" (within-host AUC 0.557 = chance)
- ✖ "The OT/internal-asset reframe is validated" (LANL is that setting and detection was chance)

9. What TO Claim

- ✔ "Dual-branch FFT+BiLSTM beacon detector is a novel combination"
- ✔ "Cross-service category-transition sequence semantics" (91-study review found nobody does it)
- ✔ "We cover what EDR cannot reach — agentless/unmanaged/IoT/OT"
- ✔ "Inspector–Sentry cascade: 13.7× compression, 96.9% agreement with teacher at 5% budget"
- ✔ "The mechanism transfers across organisations (AUC −3.3%)"
- ✔ "We found our own negative and built the tooling to prove it" (honest science)

10. Key File Paths

Code

Thing	Path
Backend root	D:\1_sih26#2\netsentinel-main\netsentinel\
V2 harness	D:\1_sih26#2\netsentinel-main\v2\
Frontend	D:\1_sih26#2\netsentinel-main\frontend\src\

Thing	Path
Inspector-Sentry demo	D:\1_sih26#2\netsentinel-main\inspector_centry\App.tsx
Entry point	D:\1_sih26#2\netsentinel-main\run.py
V2 models	v2\netsentinel_v2\models.py (Inspector, Sentry, DeferralHead)
V2 training	v2\netsentinel_v2\train.py
Service Category Resolver	v2\netsentinel_v2\categories.py + zeek_loader.py
Synthetic generator	v2\netsentinel_v2\synth.py
LANL loader	v2\netsentinel_v2\lanl_loader.py
Per-host baseline	v2\netsentinel_v2\baseline.py
Capture scripts	v2\start_capture.ps1 , v2\capture_service.ps1 , v2\install_capture_task.ps1
Zeek converter	v2\zeekify.sh

Key Documents

Doc	Path
Master context	sih26\CLAUDE.md
V2 architecture	docs_deep\ARCHITECTURE_V2.md
Critique	docs_deep\CRITIQUE.md
V2 results	v2\results.json v2\shift_results.json
LANL results	v2\lanl_novelty.json (3 seeds, novelty on), v2\lanl_base.json (no novelty)
Datasets guide	v2\DATASETS.md
Capture fix	v2\CAPTURE_FIX_6SEP.md
Post-8 Sep plan	v2\POST_8SEP.md

11. How to Run

```
# Backend
python run.py
# Wait for: [>] Server ready! [OK] 6/6 models loaded

# Frontend
cd frontend && pnpm install && pnpm dev
```

```
# Open: http://localhost:5173

# V2 experiments
cd v2
python run_experiment.py --seeds 3 --hosts 300 --days 24
python shift_test.py --seeds 3
python train_real.py --data D:\capture-zeek --seeds 3

# LANL
python train_real.py --lanl --data D:\Downloads --max-hosts 500 --lanl-days 13 --seeds 3

# Local capture
.\start_capture.ps1 -Interface <n> -OutDir D:\capture -MaxGB 40
```

12. API Contract

```
GET /api/health          → { status, models_loaded, uptime }
GET /api/alerts          → [ alert_objects... ]
GET /api/stats           → { total_alerts, by_threat_class, by_severity }
POST /api/pcap/upload     → multipart file → process
POST /api/capture/start?interface=X → start live Scapy capture
POST /api/capture/stop    → stop
POST /api/simulate/{mode} → normal|ddos|dga|c2|mixed
WS   ws://localhost:8000/ws → real-time alert stream
```